jigsaw-agile-community-rules

7th place solution

Rank	7
Team	ktr
Author	ktr
Topic ID	613215
Version	Original

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613215
Retrieved with Kaggle CLI on 2026-10-21.

Thank you for hosting such an interesting competition. The scores around my position were very close, so everyone had a chance to win Gold. Luckily, I ended up in the prize range.

Overview

- Solution completely relies on train-on-test approach.
- safety content moderation pretrained models boost ensemble score.

Data Preprocess

Since the host indicated that subreddit was not involved in the annotation process, it was completely ignored.
https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/591160#3254380

I removed all duplicates using `rule-body-label` as the unique key. This significantly reduced training time. Also, not considering subreddit at all resulted in better scores (+0.007).

When taking unique values by `rule-body-label`, we can see that labels are inconsistent. I replaced the labels with the most frequent ones for training stability. Keeping entries where majority voting was not possible (ties) slightly decreased the score, so I discarded them.

This process was performed on the combined train and test data.

CV

I built a CV using the train set with `unique rule-body-label` combinations, but there was no correlation with LB at all. It was mainly used to verify that the models could be used correctly. I completely trusted the LB due to the large amount of test data.

Models

model	public	private
Qwen3-14B	0.926	0.919
phi-4	0.924	0.920
gemma-2-9b-it	0.924	0.920
shieldgemma-9b	**0.927**	**0.922**
Qwen3-8B-Guard	0.923	0.921

I swept models around 8B-14B size that could run on T4 (16GB VRAM) with QLoRA. Unfortunately, there were very few models that could exceed the 0.920 threshold.

Adding gemma2-9b-it to the ensemble greatly boosted the score. shieldgemma as a variant brought the best performance, and single best, surpassing Qwen3-14B. Related to this, Qwen3-8B-Guard also performed better than the vanilla model (0.920 -> 0.923).

Models are trained using the Unsloth framework, with two models trained simultaneously by specifying different GPU IDs. I explored training 32B with QLoRA + DDP, but couldn't implement it due to complexity requirements.

Training and inference time combined was approximately 10 hours for the four models. The fifth model, Qwen3-8B-Guard, was only used to infer two rules.

Unlock Gemma2 on vLLM

I think this is probably the most unique aspect of my solution. vLLM doesn't allow inferring Gemma2 in fp16 due to numerical instability. However, since this is simply hardcoded, it's easy to unlock by rewriting with sed.

```
sed -i  /usr/local/lib/python3.11/dist-packages/vllm/config.py
```

(vllm==0.10.0)

Even after unlocking, I was able to achieve the above reasonable scores. (probably due to the special requirement of inferring only 1 token)

Didn't Work

- Inferring only public rules with Qwen3-32B Based on my understanding of the metrics and experiments, ranking predictions per rule doesn't affect the score. So I tried inferring only public rules with a 32B model trained on train data, but the score was significantly worse compared to ensembles of other models. Is it because the prediction distribution differs from models trained with train-on-test?
- Synthetic Data Adding synthetic data worsened the score, so I discarded the idea early on. Considering that the loss for synthetic data approached zero during training, I think it failed to emulate the test distribution.

Supplemental Q&A

> **Chris Deotte · 2025-10-25T04:24:46.177000**
>
> Great job, congratulations. Did you have enough time during submission to train all these models on the test data? What percentage of test data did you use for QLORA? And did you include train data. Also did you pretrain the models and on what data?

Athar Sayed · 2025-10-25T06:06:45.393000

@kcotton21 Congragulations I also tried to train gemma on train and 5 % of test data pseudo labelling I was able to train it but while inferring there was problem of dtype I couldnt use half dtype so decided to drop this idea as I couldnt found fix and time was less what fix did you used ?

ktr · 2025-10-25T06:12:25.223000

thank you!

- I included all test data `example` and the `body` from train. From my understanding of the data, all train `example` come from the body. Removing duplicates like subreddit reduces the samples a lot. So training with 100% of the labeled data doesn't take that long.
  For reference, with this setup, phi4 finishes submission in about 240 min. Gemma2 takes around 140 min.
- For the moderation model, I used an already trained one. I didn't train it myself. Sorry for the confusion.

ktr · 2025-10-25T06:27:37.160000

thank you!

I didn't encounter such an error. I only enabled inference with vllm using sed command and set the dtype to `half`.

Manoj · 2025-10-26T04:21:51.260000

how did you infer on shieldgemma-9b, i was able to train but during inference i was getting precision error and in "float32" i would go out of memory

```
  File , line ,  create_model_config
     ModelConfig(
           ^^^^^^^^^^^^
  File , line ,  __init__
    s.__pydantic_validator__.validate_python(ArgsKwargs(args, kwargs), self_instance=s)
pydantic_core._pydantic_core.ValidationError:  validation error  ModelConfig
  Value error, The model   does  support float16. Reason: Numerical instability. Please use
bfloat16  float32 instead. [=value_error, input_value=ArgsKwargs((), {: ...attention_dtype
```

ktr · 2025-10-26T15:11:00.387000

In vllm, fp16 inference for Gemma2 is not allowed. Please modify the hardcoded section as shown in the solution.